GemBid LK

Full ML Pipeline Evaluation Report

Gem price prediction, model comparison, SHAP explainability, API deployment, and frontend integration

Prepared for evaluation - April 29, 2026

Core Summary

Area	Remember this
Dataset	21,998 cleaned gem records with type, shape, color, clarity, treatment, price, weight, and dimensions.
Target	Model learns Log_Price for stability, then returns LKR using exp().
Best model	LightGBM selected after A/B testing against XGBoost and Random Forest.
Explainability	SHAP TreeExplainer returns per-feature positive/negative impact in LKR.
Deployment	FastAPI ML service is accessed through authenticated Node/Express API and React UI.

Team responsibility map

IT24100316 Data and preprocessing | IT24103156 Optimization and validation | IT24102032 Random Forest/model training | IT24102678 SHAP explainability | IT24104249 SHAP visualization/frontend | IT24102701 AI API and integration

1. Executive Summary

The complete ML pipeline in one clear story

The AI feature predicts gemstone market value in LKR from gem family, shape, color, clarity, treatment, carat weight, and dimensions. It returns a price, an uncertainty range, the model name, and SHAP explanations so the result is transparent rather than a black-box number.

One-sentence defense

We trained tree-based regression models on 21,998 real gem records, selected the best model using validation metrics, deployed it as a FastAPI service, and integrated SHAP explanations into the React UI through the Node backend.

Layer	Technology	Why it matters
Data	CSV, pandas, RobustScaler	Reproducible dataset prep and recovery of real carat/dimension values.
Features	11 engineered/encoded features	Combines gem identity, quality, weight, and physical proportions.
Models	LightGBM, XGBoost, Random Forest	Strong tree-based choices for non-linear tabular regression.
Optimization	Optuna, 5-fold CV, RMSE/R2/MAPE	Efficient tuning and fair validation across models.
Serving	FastAPI, Pydantic, joblib	Loads artifacts once and validates prediction requests.
Integration	Express proxy, React wizard	Keeps the ML API authenticated and user-friendly.

Current result	Value	Meaning
Best comparison model	LightGBM	Selected production artifact.
Log R2	0.9592	Explains about 95.9% of log-price variance.
Log RMSE	0.2272	Average log-space prediction error.
LKR MAPE	15.20%	Approximate percentage error after converting to LKR.
Tier accuracy	89.89% comparison / 91.64% eval	Practical market-tier correctness.

2. Member-by-Member Responsibilities

Use this as your evaluation speaking order

ID	Area	Files / proof	What to say
IT24100316	Data collection, preprocessing, dataset preparation	ml/data/final_gem_data_for_train.csv; ml/train.py; ml/app/encoders.py	Explain columns, RobustScaler inverse transform, log target, data cleaning, and feature order.
IT24103156	Optimization, A/B testing, feature selection, validation	ml/hyperparameter_tuning.py; ml/evaluation/compare_models.py; model_comparison.json	Explain Optuna, 5-fold CV, LightGBM/XGBoost/RF comparison, and why LightGBM won.
IT24102032	Model development, Random Forest, hyperparameter tuning	ml/train.py; ml/hyperparameter_tuning.py	Explain RF ensemble logic, key parameters, tuning, and joblib artifacts.
IT24102678	SHAP explainability and feature importance	ml/app/model.py; ml/evaluation/evaluate.py	Explain TreeExplainer, Shapley values, positive/negative direction, and LKR impact.
IT24104249	SHAP visualization and frontend components	SHAPWaterfallChart.jsx; AIPredictor.jsx	Explain result UI, waterfall chart, ranked feature impacts, and confidence range.
IT24102701	AI API, deployment, integration	ml/app/main.py; ml.controller.js; aiPredictorService.js	Explain FastAPI, Pydantic, Node proxy, auth, CORS, and React call.

Evaluation strategy

Each member should state the responsibility, point to one file, explain one design choice, then connect to the next part of the pipeline.

3. End-to-End Architecture

Dataset to prediction UI

Training and serving are separated. Training scripts create artifacts in ml/models. FastAPI loads those artifacts once at startup. Express protects and forwards prediction requests. React displays price, confidence, and SHAP explanations.

- CSV dataset -> preprocessing -> feature engineering -> train/compare models -> save joblib artifacts -> FastAPI /predict -> Node /api/ml/predict -> React AI Predictor
- Important artifacts: best_model.pkl, best_model_name.pkl, rmse_std.pkl, feature_names.pkl, model_comparison.json
- Runtime response: predictedPrice, confidenceLow, confidenceHigh, currency, shapValues, explanation, modelUsed

Runtime response - ml/app/schema.py

```
class PredictResponse(BaseModel):
    predictedPrice:  float
    confidenceLow:   float
    confidenceHigh:  float
    currency:        str = "LKR"
    shapValues:      List[SHAPEntry]
    explanation:     str = Field(default="", description="Human-readable SHAP summary")
    modelUsed:       str = Field(default="", description="Name of the model that generated
    the prediction")
```

4. Data Collection and Preprocessing - IT24100316

What enters the ML model

The dataset is a structured CSV of gem records. It contains gem category and quality attributes, physical measurements, price in LKR, and Log_Price. The model trains on Log_Price because raw gem prices are skewed by very expensive stones.

Column	Role	Processing
Type, Shape, Color, Clarity, Treatment	Categorical features	Converted to integers and renamed to lowercase model feature names.
Weight, X, Y, Z	Scaled numeric features	Inverse transformed using robust_scaler.joblib to recover carat and millimeter values.
Price	Original LKR price	Used for evaluation and interpretation.
Log_Price	Training target	Used as y because log prices are easier to model stably.

Why RobustScaler?

Gem data has outliers: very large stones, rare dimensions, and luxury prices. RobustScaler uses median and interquartile range, so outliers affect scaling less than mean/std scaling. This project loads the saved scaler and inverse-transforms Weight, X, Y, and Z so the model uses real-world units.

Preprocessing snippet - ml/train.py

```
scaler = joblib.load(SCALER_PATH)
scaled_vals = df[["Weight", "X", "Y", "Z"]].values
original_vals = scaler.inverse_transform(scaled_vals)

df["carat_weight"] = original_vals[:, 0]
df["x"] = original_vals[:, 1]
df["y"] = original_vals[:, 2]
df["z"] = original_vals[:, 3]
df["mean_width"] = (df["x"] + df["y"]) / 2.0
df["depth_ratio"] = df["z"] / df["mean_width"]
```

5. Feature Engineering and Validation

The exact features used in training and prediction

Feature	Type	Why included
type	Categorical	Gem family changes market value.
shape	Categorical	Cut/shape affects demand and value.
color	Categorical	Major value signal for colored gemstones.
clarity	Categorical	Cleaner stones are usually more valuable.
treatment	Categorical	Treatment changes trust and price.
carat_weight	Numeric	Strongest size/value factor, non-linear.
x, y, z	Numeric	Physical length, width, depth.
mean_width	Engineered	Face-up width summary from x and y.
depth_ratio	Engineered	Proportion signal: z / mean_width.

Canonical feature order - ml/app/encoders.py

```
FEATURE_ORDER = [
    "type", "shape", "color", "clarity", "treatment",
    "carat_weight", "x", "y", "z", "mean_width", "depth_ratio"
]
```

- Rows with missing values are dropped after conversion.
- carat_weight, x, y, z, and log_price must be positive.
- depth_ratio must be finite.
- The same transformation logic appears in training, tuning, comparison, evaluation, and prediction code.

6. Model Development and Training - IT24102032

Random Forest baseline and tree-ensemble training

This is supervised regression. X is the ordered feature matrix and y is Log_Price. The training code uses an 80/20 train-test split and 5-fold cross-validation for model stability.

Training split and CV - ml/train.py

```
X = df[FEATURE_ORDER].values
y = df["log_price"].values
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

kf = KFold(n_splits=5, shuffle=True, random_state=42)
neg_mse = cross_val_score(model, X_train, y_train, cv=kf,
                          scoring="neg_mean_squared_error", n_jobs=-1)
rmse_scores = np.sqrt(-neg_mse)
```

Why Random Forest?

Random Forest trains many decision trees with randomness and averages their outputs. It is a strong baseline for tabular data because it handles non-linear relationships and reduces overfitting compared with a single tree.

Random Forest configuration - ml/train.py

```
RandomForestRegressor(
    n_estimators=500,
    max_depth=12,
    min_samples_split=5,
    min_samples_leaf=2,
    max_features="sqrt",
    random_state=42,
    n_jobs=-1,
)
```

7. Model Optimization and A/B Testing - IT2410315

How algorithms were compared

A/B testing here means training different algorithms on the same features and same validation setup, then comparing metrics. The tested models are LightGBM, XGBoost, and Random Forest.

Model	Log R2	Log RMSE	LKR MAPE	Tier Acc
LightGBM	0.9592	0.2272	15.20%	89.89%
XGBoost	0.9583	0.2297	15.49%	89.89%
RandomForest	0.9444	0.2653	17.81%	88.61%

Why LightGBM won

LightGBM had the best log-space R2, lowest log-RMSE, lowest LKR MAPE, and tied top tier accuracy in the comparison run.

Best model selection - ml/train.py

```
best_name = max(results, key=lambda n: (results[n]["r2"], -results[n]["rmse"]))
joblib.dump(best_model, os.path.join(MODEL_DIR, "best_model.pkl"))
joblib.dump(best_name, os.path.join(MODEL_DIR, "best_model_name.pkl"))
```

8. Hyperparameter Tuning with Optuna

Why Optuna differs from traditional grid search

Hyperparameters are settings chosen before model training, such as number of trees, max depth, learning rate, sampling ratios, and regularization. The project uses Optuna to minimize mean 5-fold log-space RMSE.

Optuna objective - ml/hyperparameter_tuning.py

```
params = {
    "n_estimators": trial.suggest_int("n_estimators", 200, 1500),
    "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
    "max_depth": trial.suggest_int("max_depth", 3, 10),
    "subsample": trial.suggest_float("subsample", 0.5, 1.0),
    "reg_lambda": trial.suggest_float("reg_lambda", 1e-8, 10.0, log=True),
}
model = XGBRegressor(**params)
return float(np.sqrt(-cross_val_score(...)).mean())
```

Grid Search	Optuna
Tests every fixed combination, even weak regions.	Suggests trials intelligently and focuses on promising regions.
Gets expensive as parameter count grows.	Efficient for large continuous/integer ranges.
Candidate values must be manually listed.	Can sample ranges like 0.01 to 0.3 learning_rate.
Often wastes compute.	Can save study history and support pruning patterns.

Answer to remember

We used Optuna because grid search becomes too slow with many hyperparameters. Optuna explores the search space efficiently while optimizing our objective: mean 5-fold log-RMSE.

9. Evaluation and Feature Importance

How model quality is measured

Metric	Meaning	Why useful
MAE	Average absolute error	Easy to explain as average prediction gap.
RMSE	Square-root mean squared error	Punishes large mistakes.
R2	Variance explained	Higher is better; 1.0 is perfect.
MAPE	Percentage error	Useful because prices range from hundreds to millions.
Tier accuracy	Correct market band	Matches business use: Budget/Affordable/Mid/Premium/Luxury.

Top feature importance values

Rank	Feature	Importance
1	carat_weight	4046.0
2	x	2488.0
3	color	2338.0
4	depth_ratio	2194.0
5	mean_width	2190.0
6	y	1981.0

- Carat weight being highest matches real valuation logic.
- Dimensions and proportions matter because size and cut influence value.
- Feature importance is global; SHAP explains one specific prediction.

10. SHAP Explainability - IT24102678

How the model explains a single prediction

SHAP means SHapley Additive exPlanations. It estimates how much each feature contributes to a prediction compared with a baseline. TreeExplainer is efficient for LightGBM, XGBoost, and Random Forest.

SHAP setup - ml/app/model.py

```
_model = joblib.load(_BEST_MODEL_PATH)
_model_name = joblib.load(_BEST_NAME_PATH)
_explainer = shap.TreeExplainer(_model)

log_pred = float(_model.predict(X)[0])
predicted_price = math.exp(log_pred)
shap_values = _explainer.shap_values(X)[0]
```

- Positive contribution means the feature pushed the price up.
- Negative contribution means the feature pulled the price down.
- SHAP is local: it explains the current prediction, not just global model behavior.
- It explains the model, not guaranteed real-world causality.

LKR impact conversion - ml/app/model.py

```
def _impact_lkr(shap_val: float) -> float:
    return predicted_price - math.exp(max(log_price - shap_val, -50))

SHAPEntry(
    feature=display_name,
    value=display_val,
    direction="positive" if contrib > 0 else "negative",
    impactLkr=round(impact, 2),
)
```

11. SHAP Visualization and Frontend UX - IT241042

Turning explanations into a user-friendly chart

The frontend result screen shows predicted price, confidence range, model name, explanation sentence, and a SHAP waterfall-style chart. The chart sorts the largest impacts first and uses green/red direction colors.

Waterfall logic - SHAPWaterfallChart.jsx

```
const maxAbsImpact = Math.max(...shapValues.map((s) => Math.abs(s.impactLkr || 0)), 1);

{shapValues.map((item) => {
    const isPositive = item.direction === 'positive';
    const barPct = Math.abs(item.impactLkr || 0) / maxAbsImpact;
    const sign = isPositive ? '+' : '-';
    return <div>{item.feature} {sign}{fmtLKR(item.impactLkr)}</div>;
})}
```

- Wizard steps collect gem type, shape, carat, dimensions, clarity, color, and treatment.
- Validation blocks incomplete or invalid input before API call.
- Result page shows predictedPrice plus confidenceLow/confidenceHigh.
- Listing prefill lets valuation flow into direct sale or auction creation.

React service call - aiPredictorService.js

```
const { data } = await api.post('/ml/predict', {
    gemFamily, shape, caratWeight, clarity, color, treatment,
    x: parseFloat(x), y: parseFloat(y), z: parseFloat(z),
});
return data.data ?? data;
```

12. AI API and Integration - IT24102701

FastAPI ML service plus authenticated Node proxy

The ML model is served separately from the Node application. This keeps Python ML dependencies isolated while the Express backend remains the authenticated gateway for the React frontend.

FastAPI service - ml/app/main.py

```
@app.get("/health")
def health():
    return {"status": "ok", "model_loaded": gem_model._model is not None}

@app.post("/predict", response_model=PredictResponse)
def predict(req: PredictRequest):
    return gem_model.predict(req)
```

Express proxy - backend/src/modules/ml/ml.controller.js

```
const ML_SERVICE_URL = process.env.ML_SERVICE_URL || 'http://localhost:8000';

const mlResponse = await fetch(`${ML_SERVICE_URL}/predict`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ gemFamily, shape, color, clarity, treatment,
                           caratWeight: caratNum, x: xNum, y: yNum, z: zNum }),
});
```

- Pydantic validates request fields and numeric ranges.
- CORS origins are configured for local frontend/backend development.
- /api/ml/predict requires authentication to reduce abuse.
- 503 errors are returned when the model or service is not ready.

13. Runtime Prediction Flow

What happens after the user clicks Predict Price

- React wizard sends form data to /api/ml/predict.
- Express validates required fields, parses numbers, and forwards to FastAPI /predict.
- FastAPI maps labels to canonical integer encodings.
- mean_width and depth_ratio are computed the same way as training.
- The model predicts log_price; math.exp converts the result to LKR.
- confidenceLow and confidenceHigh are computed in log-space and exponentiated.
- SHAP values are calculated, converted to LKR impacts, sorted, and returned to React.

Request encoding - ml/app/model.py

```
row = {
    "type": TYPE_MAP[gem_family_lower],
    "shape": SHAPE_MAP[req.shape],
    "color": COLOR_MAP[req.color],
    "clarity": CLARITY_MAP[req.clarity],
    "treatment": TREATMENT_MAP[req.treatment],
    "carat_weight": req.caratWeight,
    "x": req.x, "y": req.y, "z": req.z,
    "mean_width": mean_width,
    "depth_ratio": depth_ratio,
}
return pd.DataFrame([row], columns=_feature_order)
```

How to explain confidence

It is an approximate uncertainty band based on validation error in log-space, converted back to LKR. It communicates uncertainty instead of pretending the prediction is exact.

14. Technology Stack

What we use and why

Technology	Where used	Reason
pandas / NumPy	Data preparation	Fast tabular and numeric processing.
scikit-learn	split, CV, metrics, RandomForest	Standard validation and baseline ML toolkit.
XGBoost	Candidate model	Strong boosted trees for tabular regression.
LightGBM	Best model	Fast, accurate gradient boosting on tabular data.
Optuna	Tuning	Efficient hyperparameter search over large ranges.
SHAP	Explainability	Feature-level local explanations.
joblib	Artifacts	Saves and loads Python ML models efficiently.
FastAPI + Pydantic	ML API	Python-native serving with validation schemas.
Express/Node	Backend proxy	Authenticated gateway to ML service.
React	Predictor UI	Interactive wizard and SHAP explanation display.

Why log-price and tree models?

Log_Price reduces the impact of extreme luxury stones and makes errors more proportional. Tree ensembles are used because gem prices are non-linear: the impact of carat weight depends on type, color, clarity, and treatment.

15. Speaking Notes - Data and Optimization

For IT24100316 and IT24103156

IT24100316

- My part was making sure the ML model receives clean, consistent, trainable data.
- Dataset size is 21,998 records with 11 model features.
- Raw columns include Type, Shape, Color, Clarity, Treatment, Weight, X, Y, Z, Price, Log_Price.
- RobustScaler inverse transform recovers original carat/dimension values.
- mean_width and depth_ratio are engineered because gem proportions influence value.

IT24103156

- My part was comparing algorithms and improving reliability through validation.
- A/B testing means same dataset, same split, different algorithms.
- Candidates were XGBoost, Random Forest, and LightGBM.
- LightGBM achieved Log R2 = 0.9592 and Log RMSE = 0.2272.
- Optuna is smarter than grid search because it samples useful ranges instead of blindly testing every fixed combination.

16. Speaking Notes - Model and SHAP

For IT24102032 and IT24102678

IT24102032

- My part was implementing and training regression models, especially Random Forest as a strong baseline.
- Random Forest averages many trees to reduce overfitting.
- Important parameters: n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features.
- Even though LightGBM won, Random Forest is a reliable baseline for comparison.
- Model artifacts are saved with joblib for production use.

IT24102678

- My part was making the model explainable, not just accurate.
- SHAP gives each feature a contribution value.
- TreeExplainer is optimized for tree models.
- Positive SHAP values push price up; negative values pull price down.
- Because the model predicts log price, we convert contribution into approximate LKR impact for users.

17. Speaking Notes - Frontend and API

For IT24104249 and IT24102701

IT24104249

- My part was turning ML output into a clear user experience.
- The wizard collects gem type, shape, carat, dimensions, color, clarity, and treatment.
- The result screen shows price, confidence range, model name, and SHAP explanation.
- The waterfall chart ranks the biggest impacts and uses direction colors.
- The predicted price can prefill listing creation for sale or auction.

IT24102701

- My part was making the trained model usable by the real web application.
- FastAPI loads ML artifacts once and exposes /health and /predict.
- Pydantic validates request data before prediction.
- Express authenticates users and forwards to FastAPI through ML_SERVICE_URL.
- If the ML service is unavailable, the backend returns 503 instead of fake predictions.

18. Evaluation Q&A Cheat Sheet

Short answers to likely questions

Question	Best answer
Why Log_Price?	Gem prices are skewed; log transform stabilizes training and exp converts back to LKR.
Why RobustScaler?	It uses median/IQR so outliers affect scaling less than StandardScaler.
Why Optuna?	Grid search blindly tests fixed combinations; Optuna searches ranges efficiently.
Why LightGBM?	It had the best comparison metrics: highest R2 and lowest RMSE/MAPE.
Why SHAP?	It explains each prediction by showing which features pushed the price up or down.
What is confidence range?	Approximate uncertainty from validation log-RMSE converted back to LKR.
How avoid feature mismatch?	Training and prediction both use FEATURE_ORDER from encoders.py.
Why FastAPI plus Express?	FastAPI serves Python ML; Express keeps the authenticated app API gateway.
Is this a professional appraisal?	No, it is a data-driven estimate for platform guidance.

19. Commands to Run and Verify

Useful for demo or evaluator questions

Training

```
cd ml
python train.py
# Saves best_model.pkl, best_model_name.pkl, rmse_std.pkl, feature_names.pkl,
    model_comparison.json
```

Hyperparameter tuning

```
cd ml
python hyperparameter_tuning.py
# Runs Optuna studies for XGBoost, RandomForest, and LightGBM.
```

Evaluation

```
cd ml
python evaluation/compare_models.py
python evaluation/evaluate.py
# Generates JSON metrics and plots for comparison, residuals, feature importance, SHAP, and
    tiers.
```

ML API

```
cd ml
uvicorn app.main:app --reload --port 8000
# GET /health checks model loading.
# POST /predict returns price, confidence range, SHAP values, explanation, and model name.
```

Demo tip

First open /health to prove the model is loaded, then submit one prediction through the React AI Predictor to show price plus SHAP explanation end to end.

20. Final Recap

The narrative to remember

GemBid LK has a complete production-style ML flow: prepared data, engineered features, multiple model comparison, best-model artifact saving, SHAP explanations, FastAPI serving, Node authentication/proxying, and React visualization.

- We compare XGBoost, Random Forest, and LightGBM instead of using one arbitrary model.
- We train on Log_Price for stability and convert back to LKR for users.
- FEATURE_ORDER and encoders.py keep training and inference consistent.
- Optuna is more efficient than grid search for complex tuning spaces.
- SHAP makes the valuation explainable by showing feature-level price impact.
- FastAPI separates Python ML serving from the Node web backend.

Final one-liner

GemBid LK uses a validated, explainable ML pipeline: LightGBM predicts gem prices, SHAP explains the prediction, FastAPI serves it, Express secures it, and React presents it clearly to the user.